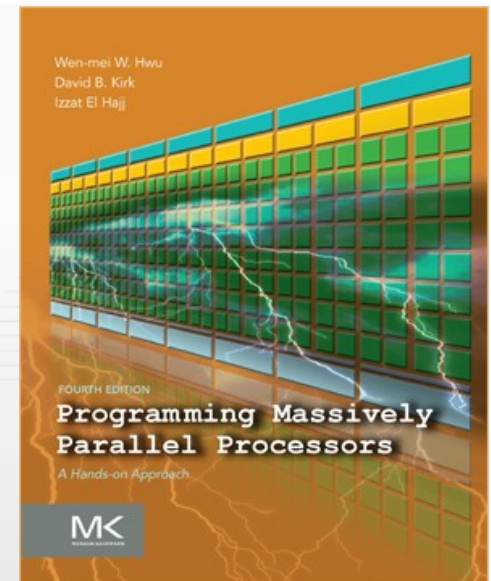
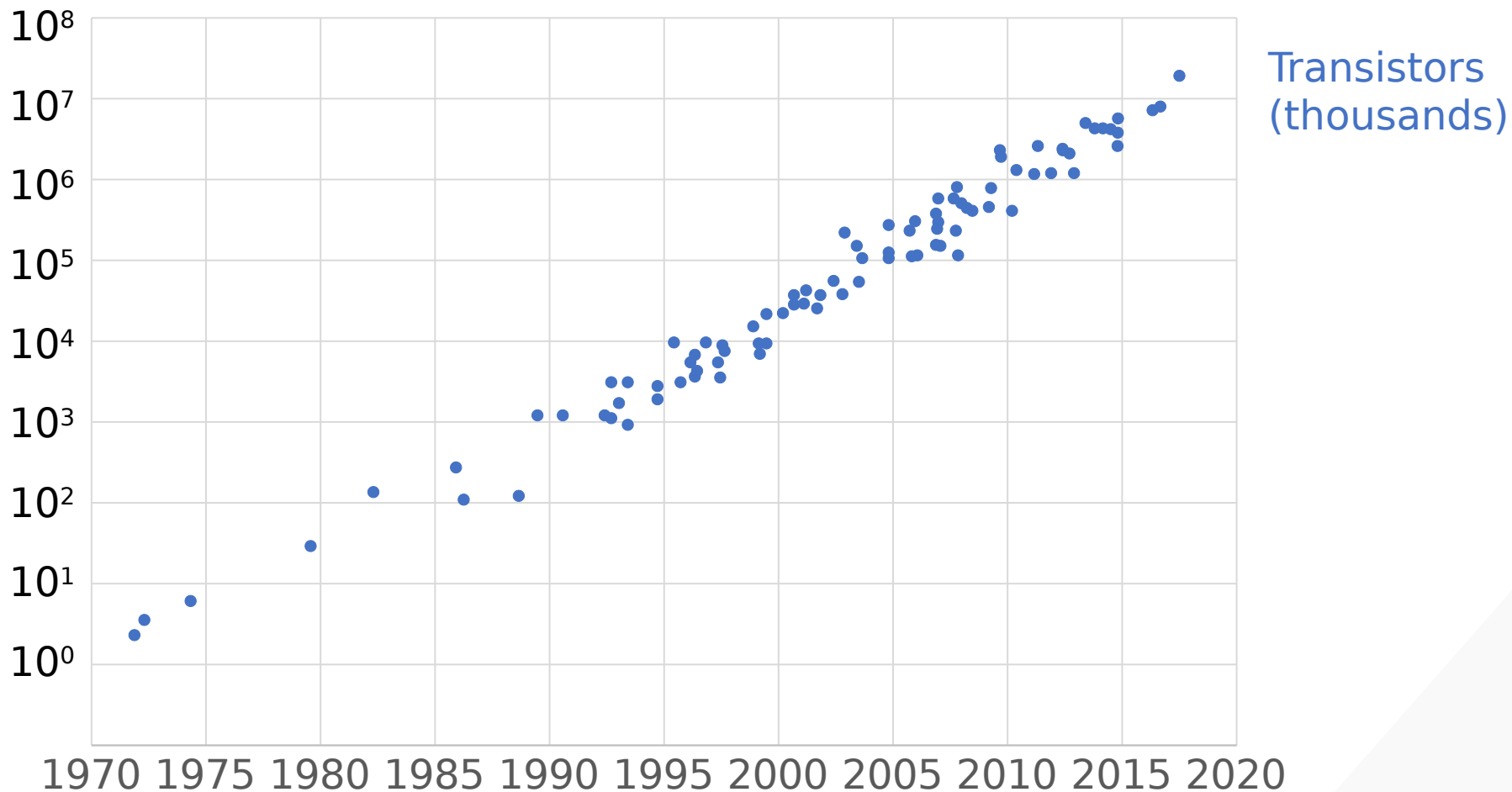


Programming Massively Parallel Processors

A Hands-on Approach

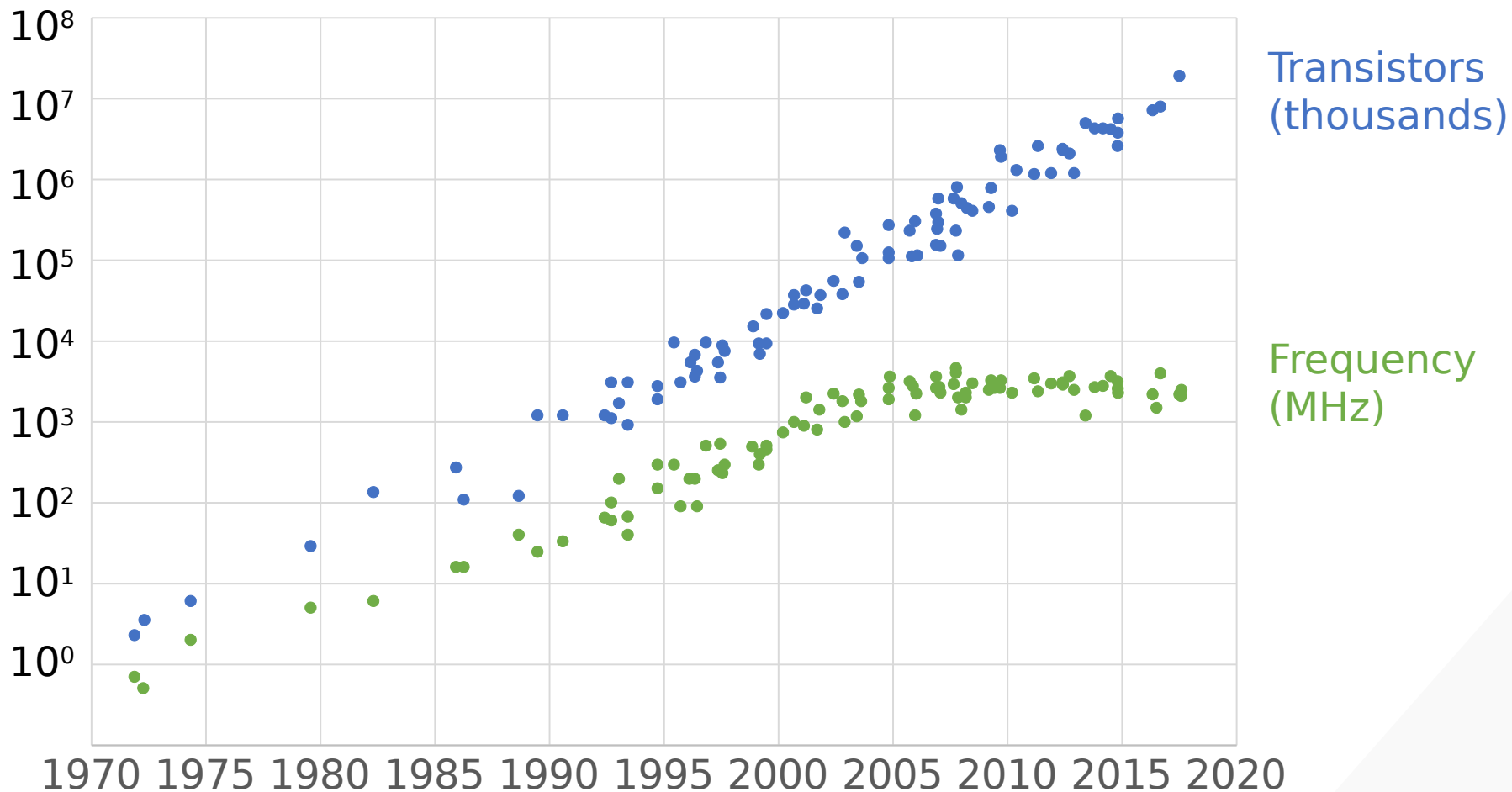
CHAPTER 1 > Introduction





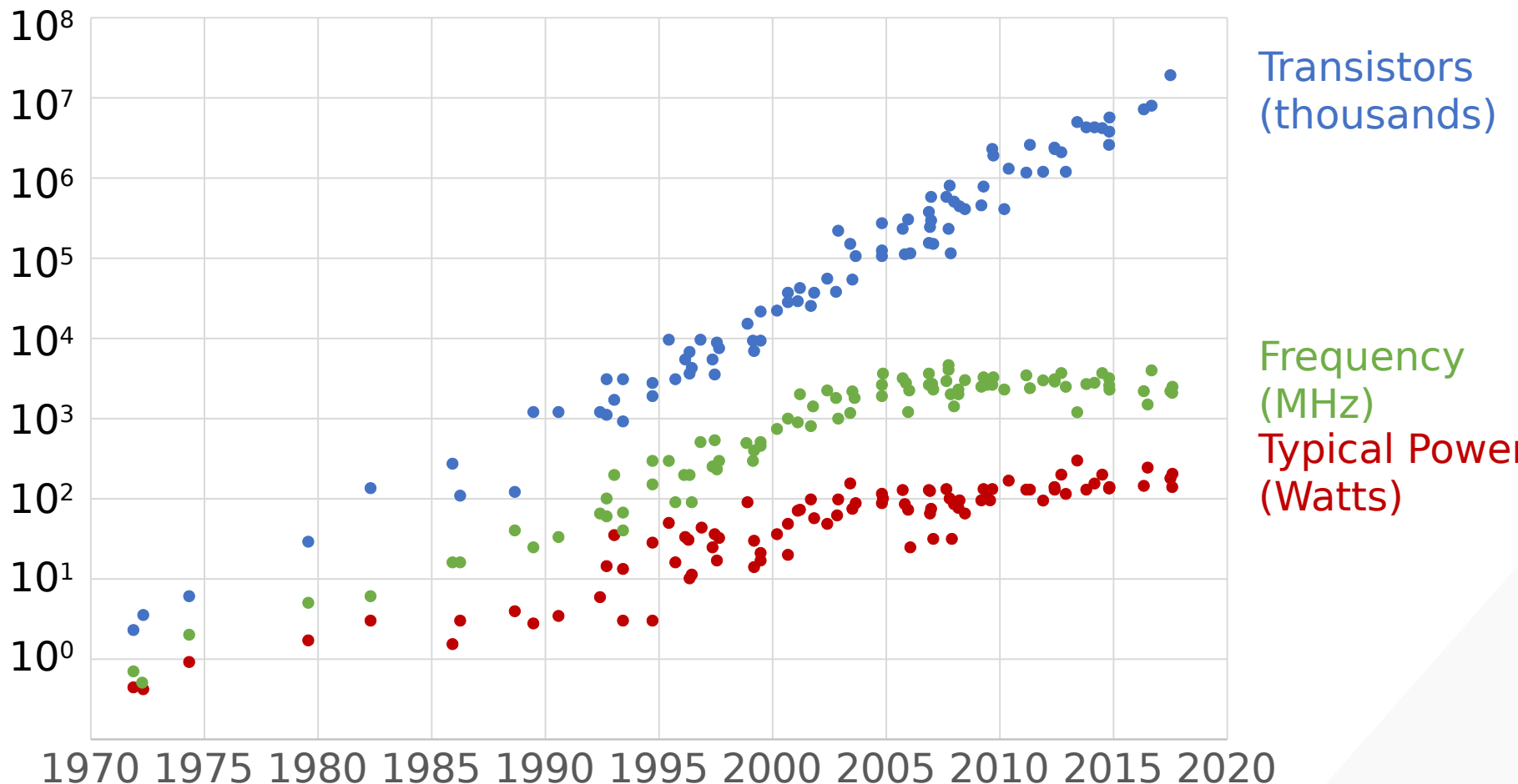
Source: M. Horowitz, F. Labonte, O. Shacham, K. Olukotun, L. Hammond, C. Batten (1970-2010). K. Rupp (2010-2017).

Moore's "Law" predicted that the number of **transistors** per unit area would double every 18-24 months



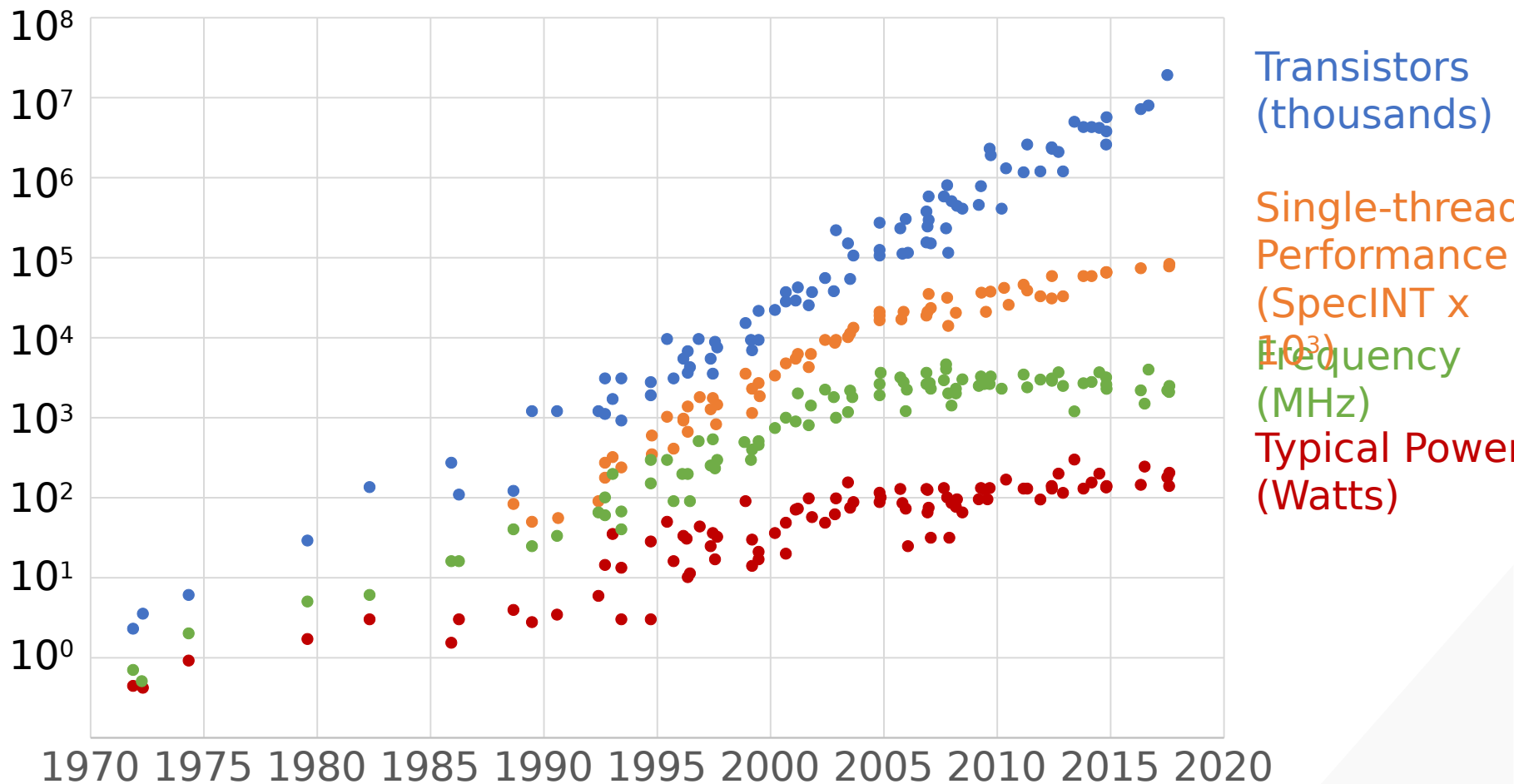
Source: M. Horowitz, F. Labonte, O. Shacham, K. Olukotun, L. Hammond, C. Batten (1970-2010). K. Rupp (2010-2017).

Processor **frequency** (clock rate) followed the same trend because smaller transistors can be switched faster... until around 2005.



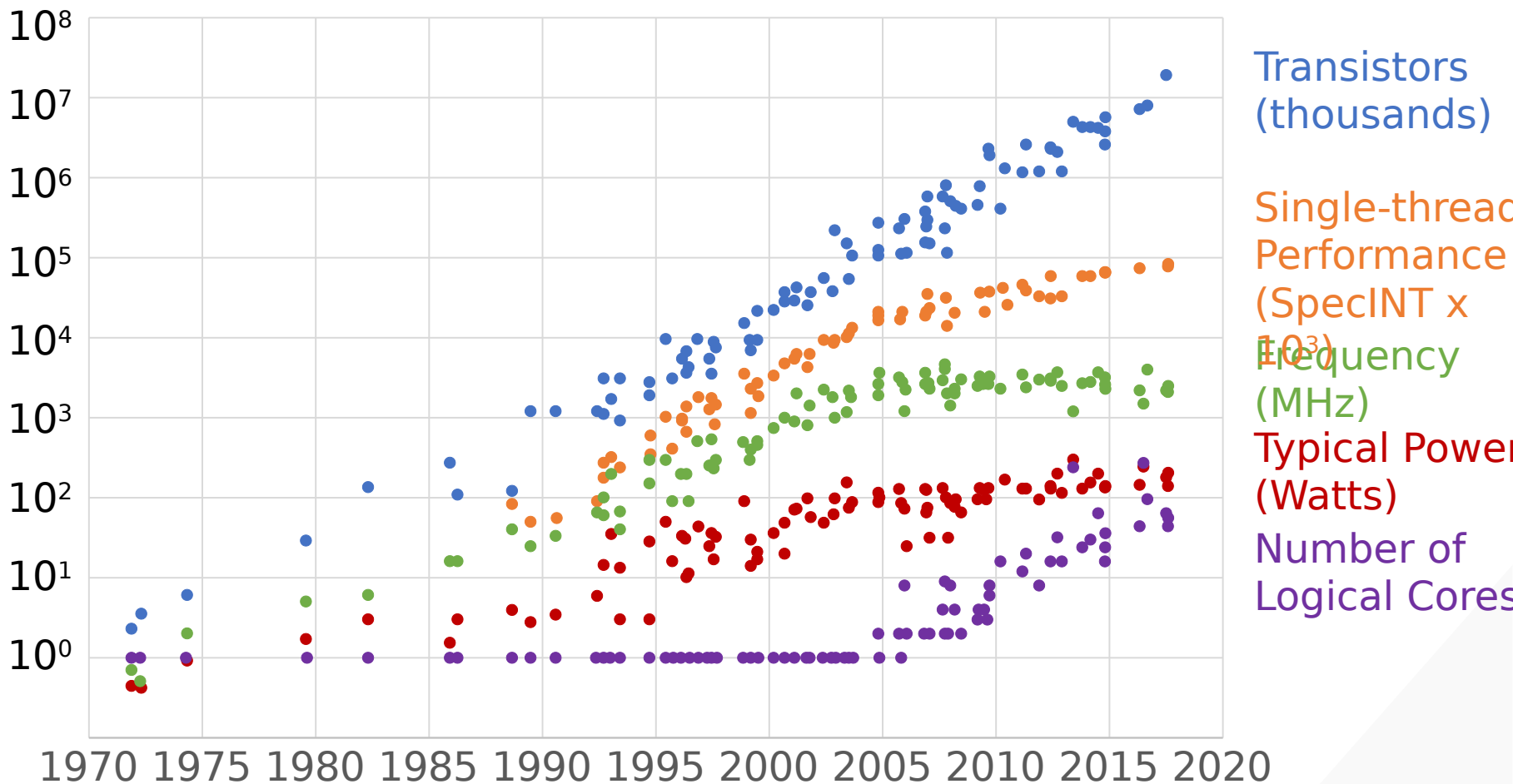
Source: M. Horowitz, F. Labonte, O. Shacham, K. Olukotun, L. Hammond, C. Batten (1970-2010). K. Rupp (2010-2017).

Around 2005, frequency stopped increasing due to the **power wall** (increasing frequency further would make the chip too hot to cool feasibly)



Source: M. Horowitz, F. Labonte, O. Shacham, K. Olukotun, L. Hammond, C. Batten (1970-2010). K. Rupp (2010-2017).

Stagnation in frequency led to a stagnation in **single-threaded performance**, except for some improvements due to architecture and compiler advancements



Source: M. Horowitz, F. Labonte, O. Shacham, K. Olukotun, L. Hammond, C. Batten (1970-2010). K. Rupp (2010-2017).

Stagnation in single-thread performance made parallel computing mainstream as transistors were invested in adding **more cores** to processors

Latency-Oriented Design



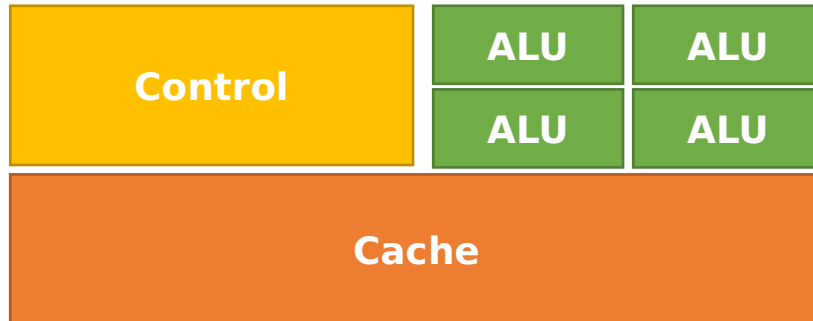
Minimize the
time it takes to
perform a
single task

Throughput-Oriented Design



Maximize the number
of tasks that can be
performed in a given
time frame

CPU: Latency-Oriented Design



ALUs

Reduced operation latency

caches

Convert long latency memory accesses to short latency cache accesses

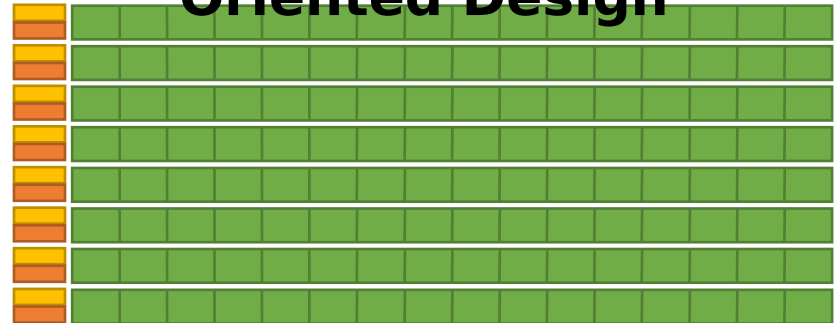
control

Branch prediction to reduce control hazards

Data forwarding to reduce data hazards

Modest multithreading
latency

GPU: Throughput-Oriented Design



ALUs

Long latency, high throughput
Heavily pipelined for further throughput

caches

More area dedicated to computation

control

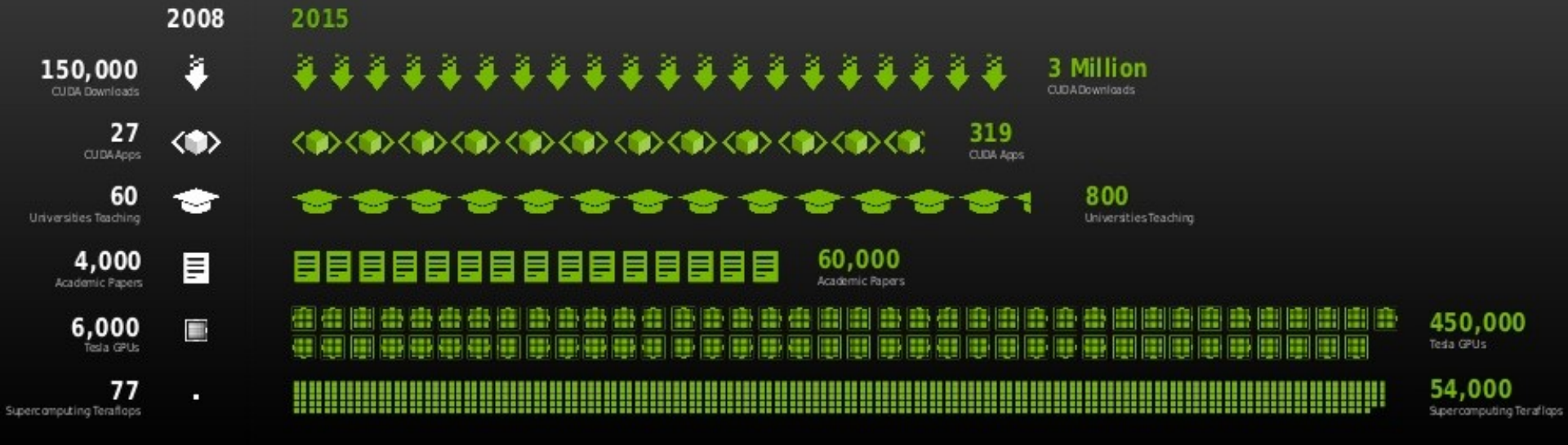
More area dedicated to computation

hide the

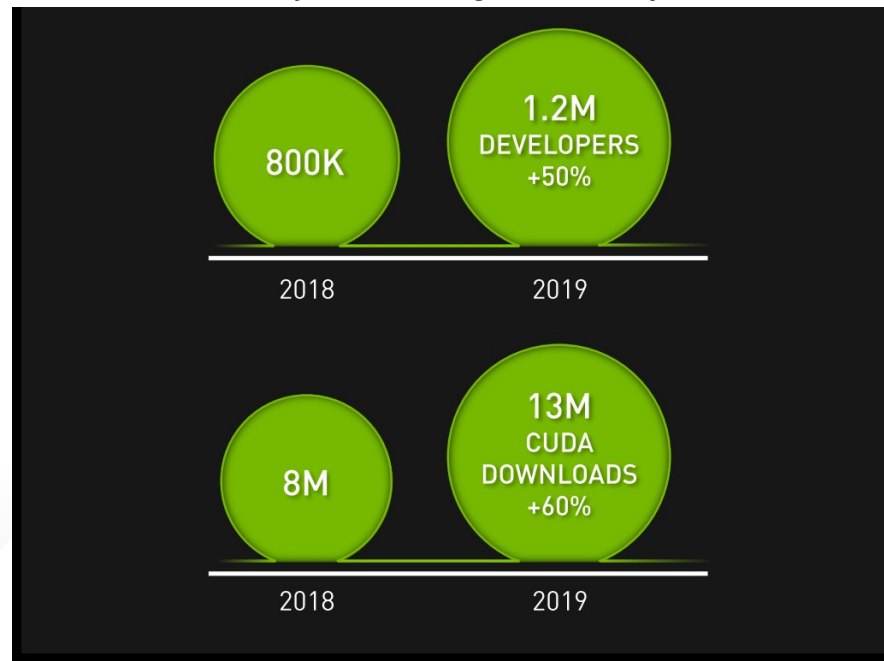
- GPUs were originally built for graphics
 - Hence the name, Graphics Processing Unit
 - Graphics workloads process a massive number of pixels in parallel



- Before 2007, the only way to program GPUs was using graphics APIs
 - e.g., OpenGL, Direct3D
 - Using GPUs for other than graphics meant reformulating a computation as a function that paints a pixel
- In 2007, NVIDIA released CUDA
 - A programming interface for using GPUs in a general purpose way
 - Requires extensions to the GPU architecture



Source: Jensen Huang, GTC'15 Keynote



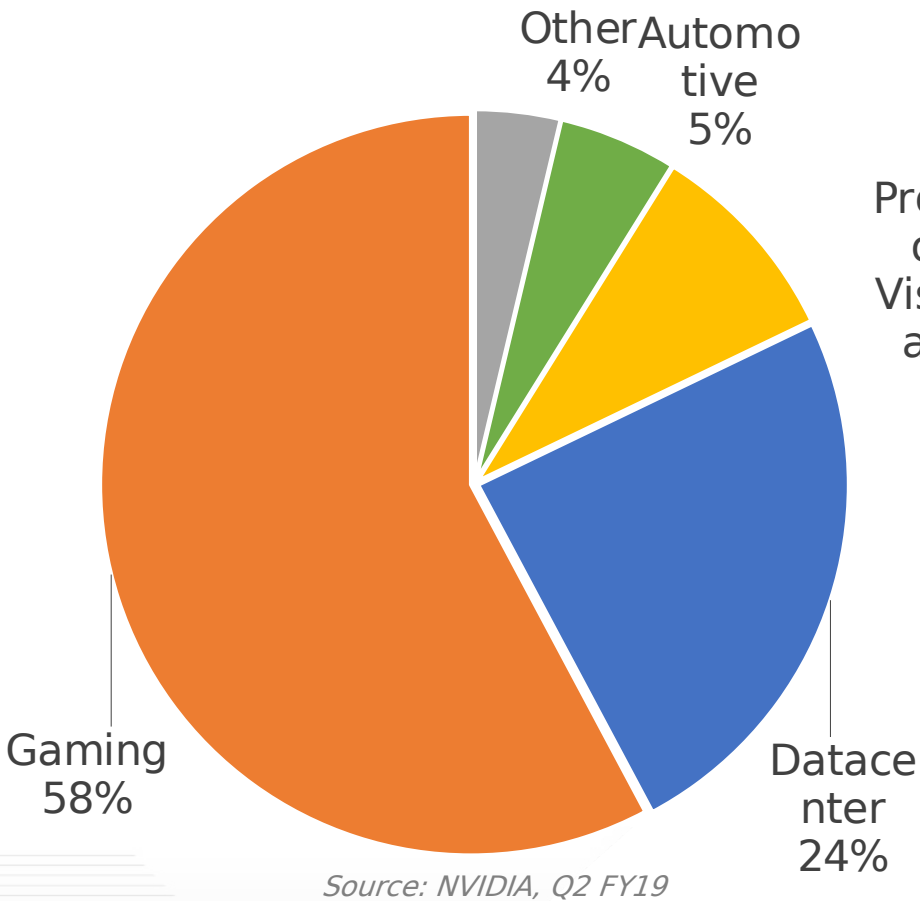
Source: Jensen Huang, GTC'19 Keynote

Source: top500.org, November 2021

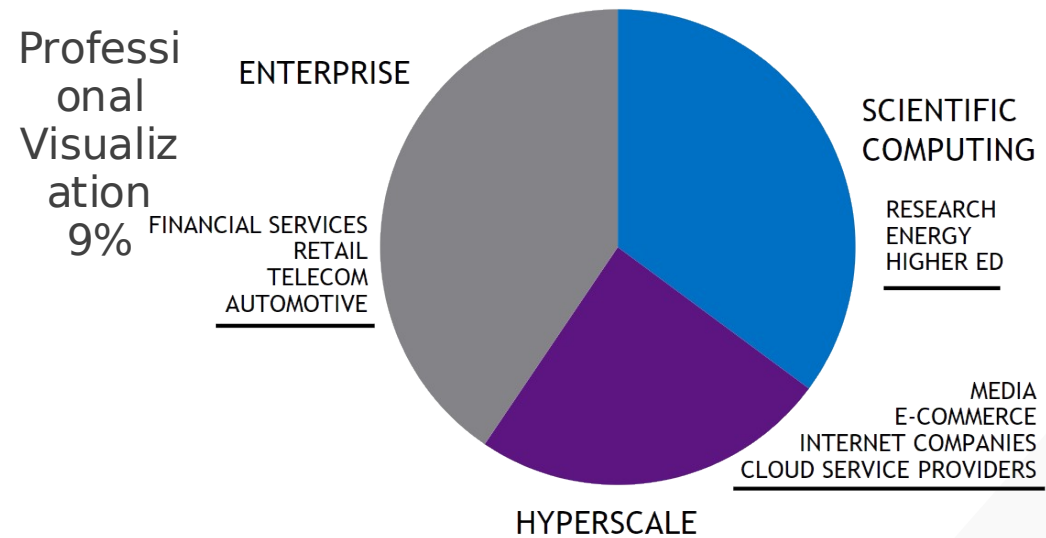
- GPU accelerated computers in 2020:
 - #1 in USA, #2 worldwide – Summit
 - #1 in Europe, #8 worldwide – Juwels
 - 7 out of the top 10 supercomputers worldwide
 - #2 in energy-efficient supercomputers (Green500)
 - 9 out of top 10 energy-efficient supercomputers

- Chips are expensive to build and require a large volume of sales to amortize the cost
 - This makes the chip market very difficult to penetrate
- When parallel computing became mainstream, GPUs already had (and still have) a large *installed base* from the gaming sector
 - Gave them a head start compared to other potential massively parallel accelerators

Revenue from Different Market Sectors



Breakdown of Datacenter Sector



Source: NVIDIA, 2018

- In this class, we will focus on CUDA
- Other GPU parallel programming frameworks:
 - OpenCL
 - OpenACC
- Popular CPU programming frameworks
 - MPI (distributed memory system)
 - OpenMP (shared memory system)
 - Both can be used in conjunction with CUDA

- Consider an application where:
 - The sequential execution time is **100s**
 - The fraction of execution that is parallelizable is **90%**
 - The speedup achieved on the parallelizable part is **1000x**
- What is the overall speedup of the application?

$$t_{parallel} = (1 - 0.9) * 100s + \frac{0.9 * 100s}{1000} = 10.09s$$

$$speedup = \frac{t_{sequential}}{t_{parallel}} = \frac{100s}{10.09s} = 9.17 \times$$

- In general, if an application has:
 - t sequential execution time
 - p fraction of execution that is parallelizable
 - s speedup achieved on the parallelizable part
- What is the overall speedup of the application?

$$t_{parallel} = (1 - p) * t + \frac{p * t}{s} = \left(1 - p + \frac{p}{s}\right) * t$$

$$speedup = \frac{t_{sequential}}{t_{parallel}} = \frac{t}{\left(1 - p + \frac{p}{s}\right) * t} = \frac{1}{1 - p + \frac{p}{s}} \xrightarrow{s \rightarrow \infty} \frac{1}{1 - p}$$

- The maximum speedup of a parallel program is limited by the fraction of execution that is parallelizable

$$\text{speedup} < \frac{1}{1 - p}$$

- e.g., if p is 90%, speedup < 10×
- Fortunately, for many real applications, $p > 99\%$ especially for large datasets, and speedups >100× are attainable

- Wen-mei W. Hwu, David B. Kirk, and Izzat El Hajj. *Programming Massively Parallel Processors: A Hands-on Approach*. Morgan Kaufmann, 2022.